TinkerWithMe

tinkerwithanne@gmail.com
Nairobi, Kenya

Robot Arm & Claw · Full Curriculum

2-session series (2 hrs each) · 12–15 yrs · Advanced

Full facilitator curriculum **PREMIUM**

Overview

Assemble and program a 3-DOF robot arm with a gripper claw and a joystick controller.

DETAIL	
Track	Arduino Robotics & Electronics
Format	2-session series (2 hrs each)
Ages	12–15 yrs
Level	Advanced
Price	from KES 7,000
Standalone	No
Prerequisites	Servo motor control (free)

Course Overview

Robot Arm & Claw is a two-session advanced build in which students construct a real 3-DOF (degree-of-freedom) robot arm — base rotation, shoulder, and elbow joints — finished with a working gripper claw, and drive it with a custom-built two-joystick controller wired to an Arduino. This is a full electromechanical project: students do precision hardware assembly, multi-servo wiring with a dedicated power supply, analog input reading, PWM servo control, calibration, and finally autonomous-feeling manual control good enough to complete a real task.

The arc runs from *hardware to intelligence*. Session 1 is almost entirely hands-on assembly and wiring: mounting three SG90 servos into an articulated arm chassis, building the joystick controller box, and running a diagnostic sketch that proves every wire is correct. Session 2 shifts to software craftsmanship: students calibrate

safe movement ranges for each joint so the arm never crashes into itself, mount and wire a fourth servo as a gripper claw, upgrade the code with smoothing and deadzones for controllable motion, and then put it all to the test in a timed pick-and-sort challenge with coloured objects.

Every student (or pair) leaves with a fully assembled, fully wired, fully programmed 3-DOF robot arm with claw and joystick remote — a genuine take-home robot, plus the calibration data and source code needed to keep tuning it at home.

Learning Outcomes

- Assemble a multi-joint servo-driven mechanism with correct horn alignment, screw torque, and range-of-motion planning
- Wire multiple SG90 servos to a shared external power rail safely, separate from the Arduino's own 5V line, with a common ground
- Read and interpret dual-axis analog joystick modules (X, Y, and pushbutton) using `analogRead()` and `digitalRead()`
- Use the Arduino `Servo` library to drive multiple PWM servos simultaneously and understand PWM pulse-width control of angle
- Calibrate mechanical safe-zones (min/max angles) per joint to prevent self-collision and servo stall/damage
- Apply deadzones, mapping, and slew-rate smoothing in code to convert noisy raw analog input into controlled, human-friendly motion
- Operate a completed robot to perform a functional task (grasp, lift, move, release) under time pressure, demonstrating fine motor coordination between hardware and control code

Materials Master List

ITEM	SPEC	QTY PER STUDENT/ GROUP	NOTES
Arduino Uno R3 (or genuine-compatible)	ATmega328P, USB-B	1	Available at Nerokas / Pixel Electronics
USB cable	USB-A to USB-B, 1 m	1	For programming and Serial Monitor use

Micro servo	SG90, 9 g, 4.8–6V	4 (base, shoulder, elbow, claw)	Buy 1–2 spares per class; SG90s are the weak point — test each before mounting
3-DOF robot arm mechanical kit	Laser-cut acrylic or MDF arm chassis with base turntable, shoulder bracket, elbow bracket, gripper/claw mechanism, servo horns, M2/M3 screws & standoffs	1 set	Common "4-servo robot arm claw kit" — confirm claw mechanism included
Analog joystick module	2-axis (X, Y potentiometers) with push-button switch, e.g. KY-023 style, 5V	2	Each has 5 pins: GND, +5V, VRx, VRy, SW
Breadboard	830-point, full size	1	For joystick controller wiring
Jumper wire pack	Male-male and male-female, assorted lengths	~40 wires	Servos already have 3-pin female headers — use male-male to bridge to Arduino
Battery holder	4 x AA holder with on/off switch	1	Use 4 x NiMH rechargeable AA (4.8V total) — matches SG90 voltage spec exactly, no regulator needed
AA rechargeable batteries	NiMH, 1.2V, 2000mAh+	4	Keep a charged spare set per class
Electrolytic capacitor	1000 µF, 16V+, radial	1	Placed across the servo power rail to smooth current spikes when servos move together
Small project box / enclosure	Plastic case, approx. 12x8x4 cm, drillable	1	Houses the two joysticks as the controller unit
Screwdriver set	Precision Phillips #0 and flat #1	1 per group	For arm kit screws and case drilling
Cable ties / hot glue gun	Small zip ties, mini glue gun + sticks	shared classroom stock	Strain-relief for servo wires and controller cable
Coloured pom-poms or foam cubes	~2 cm, red/green/blue	12 (4 of each colour)	Objects for the sorting challenge
Sorting bins/cups	Small labelled cups or bowls, 3 colours	3	Placed at fixed "zones" around the arm
Multimeter	Basic digital multimeter	1 per class (facilitator)	

			To verify battery pack voltage and rail continuity before powering servos
Masking tape & marker	—	1 roll + 1 marker per group	Labelling wires and calibration values

Session Plans

Session 1: Build & Wire

Objectives

- Mechanically assemble a 3-DOF robot arm chassis using three SG90 servos as the base, shoulder, and elbow joints
- Build a two-joystick controller and wire it plus all three arm servos correctly to an Arduino and an external battery pack
- Run and interpret a diagnostic sketch to confirm every wire and joint works before moving to programming

Timing (120 minutes)

- 0–10 min: Welcome, safety briefing (servo stall heat, battery polarity, no forcing joints by hand), overview of the finished robot
- 10–25 min: Explain how a servo works (PWM pulse → angle), pass around a servo, explain why external power is needed for multiple servos
- 25–70 min: Guided mechanical assembly of the arm — base turntable and base servo, shoulder bracket and servo, elbow bracket and servo, horns screwed on at centre (90°) position
- 70–90 min: Wire the three servos to the Arduino signal pins and to the shared battery power rail; tie all grounds together; add smoothing capacitor
- 90–105 min: Build the joystick controller — mount both joystick modules in the project box, wire VCC/GND/VRx/VRy/SW to the breadboard and Arduino
- 105–115 min: Upload the wiring-test sketch, verify each joystick axis drives the correct servo, troubleshoot any reversed or dead wires
- 115–120 min: Clean workstations, label wires with tape, preview Session 2 (calibration + claw + challenge)

Step-by-step

1. Open with a live demo: hold up an SG90, connect it briefly to 5V/GND/signal on a test Arduino, and sweep it with a simple `servo.write()` command so students see cause and effect before touching their own kits.
2. Explain the "3 DOF + claw" concept on a whiteboard: base rotates left/right, shoulder tilts the whole arm up/down, elbow bends the forearm, and the claw (added next session) opens/closes at the end.
3. Hand out arm kits. Have students dry-fit all acrylic/MDF pieces without screws first to understand the assembly order.
4. Guide screw-by-screw: attach the base servo into the turntable mount, then the shoulder bracket onto the base servo horn, then the shoulder servo into that bracket, then the elbow bracket onto the shoulder servo horn, then the elbow servo. Insist every servo horn is attached at the servo's centred (90°) position — power the servo briefly to 90° with a spare Arduino/test rig before screwing on each horn, so the arm starts life balanced in its mid-range.
5. Do NOT attach the claw mechanism yet — leave the elbow's output horn bare; it will be fitted with the claw in Session 2.
6. Move to wiring: explain that 3–4 servos moving together can pull more current than USB/Arduino 5V can safely supply, causing resets — this is why a separate 4xAA battery pack powers the servos, while the Arduino keeps its own USB power for logic and signal.
7. Wire the three servo signal wires to D9 (base), D10 (shoulder), D11 (elbow). Wire all three servo red (power) wires to the battery pack's positive rail via the breadboard, and all three brown/black (ground) wires to a shared ground rail. Critically, connect that ground rail to an Arduino GND pin too, so signal and power share a common reference.
8. Solder/clip the 1000 µF capacitor across the battery rail (+ to +, − to −) near the servos to absorb current spikes.
9. Use the multimeter to confirm the battery rail reads approximately 4.8–5V before connecting any servo — this is a facilitator safety checkpoint, do not skip it.
10. Build the joystick controller: mount both joystick modules into the project box holes, wire each one's VCC to Arduino 5V, GND to Arduino GND, VRx/VRy to A0/A1 (joystick 1) and A2/A3 (joystick 2), and SW pins to D2 and D3.

11. Upload the Session 1 wiring-test sketch. Open Serial Monitor at 9600 baud. Have each student move each joystick axis and watch (a) the printed numbers change and (b) the correct servo move. Fix any mismatched pins now.
12. Label every wire at both ends with small pieces of masking tape (e.g. "BASE SIG", "SHLD SIG") — this saves enormous time in Session 2.

Build detail

Materials for this session: Arduino Uno, USB cable, 3x SG90 servo (base, shoulder, elbow — claw servo stays in its bag until Session 2), 3-DOF arm mechanical kit, 2x analog joystick modules, breadboard, jumper wires, 4xAA battery holder + batteries, 1000 µF capacitor, project box, screwdriver set, cable ties, masking tape.

Wiring / assembly list (component → pin):

- Base servo signal (orange/yellow wire) → Arduino D9
- Shoulder servo signal → Arduino D10
- Elbow servo signal → Arduino D11
- All three servo red (power) wires → breadboard positive rail → battery pack (+) via switch
- All three servo brown/black (ground) wires → breadboard negative rail → battery pack (−)
- Breadboard negative rail → Arduino GND (common ground link — essential)
- 1000 µF capacitor → across breadboard positive/negative rail, close to servos
- Joystick 1: VCC → Arduino 5V, GND → Arduino GND, VRx → A0, VRy → A1, SW → D2
- Joystick 2: VCC → Arduino 5V, GND → Arduino GND, VRx → A2, VRy → A3, SW → D3

```
/*
  Session 1: Wiring Test Sketch
  Robot Arm & Claw - TinkerWithMe

  Purpose: prove every wire is correct before we do any real programming.
  - Centres all three servos at start (90 degrees) - if a servo doesn't
    move to centre, its signal or power wiring is wrong.
  - Continuously prints raw joystick readings AND directly drives the
    servos from the joysticks (no calibration yet - that is Session 2).
*/

#include
```

```
// ---- Pin assignments ----
const int PIN_JOY1_X = A0;   // will drive Base
const int PIN_JOY1_Y = A1;   // will drive Shoulder
const int PIN_JOY2_X = A2;   // will drive Elbow
const int PIN_JOY2_Y = A3;   // not used yet - reserved for claw in Session 2
const int PIN_JOY1_SW = 2;   // not used yet
const int PIN_JOY2_SW = 3;   // will control claw in Session 2

const int PIN_SERVO_BASE     = 9;
const int PIN_SERVO_SHOULDER = 10;
const int PIN_SERVO_ELBOW    = 11;

Servo baseServo;
Servo shoulderServo;
Servo elbowServo;

void setup() {
  Serial.begin(9600);

  baseServo.attach(PIN_SERVO_BASE);
  shoulderServo.attach(PIN_SERVO_SHOULDER);
  elbowServo.attach(PIN_SERVO_ELBOW);

  pinMode(PIN_JOY1_SW, INPUT_PULLUP);
  pinMode(PIN_JOY2_SW, INPUT_PULLUP);

  // Centre every servo first - this is our wiring proof.
  baseServo.write(90);
  shoulderServo.write(90);
  elbowServo.write(90);
  delay(1000);

  Serial.println("Wiring test running. Move each joystick and watch the numbers + the arm.");
}

void loop() {
  int j1x = analogRead(PIN_JOY1_X);
  int j1y = analogRead(PIN_JOY1_Y);
  int j2x = analogRead(PIN_JOY2_X);
  int j2y = analogRead(PIN_JOY2_Y);
  int sw1 = digitalRead(PIN_JOY1_SW);
  int sw2 = digitalRead(PIN_JOY2_SW);

  // Direct, un-calibrated mapping just so the arm visibly responds today.
  // Session 2 replaces this with safe, calibrated, smoothed control.
  int baseAngle     = map(j1x, 0, 1023, 0, 180);
  int shoulderAngle = map(j1y, 0, 1023, 0, 180);
  int elbowAngle    = map(j2x, 0, 1023, 0, 180);

  baseServo.write(baseAngle);
  shoulderServo.write(shoulderAngle);
  elbowServo.write(elbowAngle);

  // Print everything so students can confirm each wire goes to the right pin.
  Serial.print("J1 X:"); Serial.print(j1x);
  Serial.print(" Y:");   Serial.print(j1y);
  Serial.print(" SW:");  Serial.print(sw1);
  Serial.print(" | J2 X:"); Serial.print(j2x);
  Serial.print(" Y:");      Serial.print(j2y);
  Serial.print(" SW:");     Serial.print(sw2);
```

```
  Serial.print(" | Angles B:"); Serial.print(baseAngle);
  Serial.print(" S:");          Serial.print(shoulderAngle);
  Serial.print(" E:");          Serial.println(elbowAngle);

  delay(100);
}
```

Checks for understanding

- Why do we power the servos from a separate battery pack instead of the Arduino's 5V pin?
- What would happen if we forgot to connect the battery ground to the Arduino ground?
- If moving Joystick 1 left makes the base servo turn right, how would you fix that in code without rewiring?

Common mistakes & fixes

MISTAKE	SYMPTOM	FIX
Servo horn attached before centring servo	Arm sits crooked or hits its own frame at 90°	Detach horn, power servo alone to 90° first, then reattach horn straight
Grounds not shared between battery and Arduino	Servos jitter erratically or don't respond at all	Run one extra jumper from the breadboard ground rail to an Arduino GND pin
Joystick wired to wrong analog pin	Wrong servo moves, or numbers don't change on Serial Monitor	Trace wire physically from module to pin, relabel with tape, fix in wiring not code
Too many servos drawing power at once with weak batteries	Arduino resets/restarts when servos move together	Confirm battery voltage with multimeter, use fresh/charged NiMH cells, check capacitor is in place

Session 2: Program & Challenge

Objectives

- Calibrate safe minimum/maximum movement ranges for each joint to prevent self-collision
- Mount and wire a claw/gripper servo and add smoothed, deadzone-filtered control code
- Complete a timed challenge: pick up coloured objects and sort them into the correct coloured zone

Timing (120 minutes)

- 0–10 min: Recap Session 1, state today's goal (safe calibrated arm + working claw + challenge)
- 10–30 min: Calibration exercise using the helper sketch and a worksheet to record each joint's safe min/max angle
- 30–45 min: Mount the claw mechanism onto the elbow's output horn, attach the claw servo, wire it to D6
- 45–75 min: Update the sketch with calibrated constants, claw control, deadzone and smoothing; upload and test
- 75–90 min: Free practice — pick up small objects, adjust calibration values if the arm still collides or feels too twitchy
- 90–110 min: Sorting challenge in timed rounds with scoring
- 110–120 min: Showcase, reflection, pack robots for take-home

Step-by-step

1. Recap the previous session's wiring test and remind students the current code has no safety limits — moving the joystick fully can crash the arm into itself.
2. Introduce calibration: explain that every physical build is slightly different (screw tightness, horn position), so each joint's safe range must be found by testing, not assumed.
3. Upload the calibration helper sketch below. Using Serial Monitor commands, have students test each joint's extremes slowly, watching for the arm touching its own frame or the table, and record the safe MIN and MAX angle for base, shoulder, and elbow on their worksheet.
4. Once all three ranges are recorded, move to the claw. Attach the claw/gripper mechanism to the elbow's bare output horn. Attach the 4th SG90 as the claw servo per the kit's gripper linkage instructions.
5. Wire the claw servo signal to D6, and its power/ground into the same shared rail as the other three servos.
6. Test the claw servo alone first: temporarily write a tiny sketch (or use the calibration helper) to find the claw's fully-open angle and fully-closed (gripping, not stalling) angle — write these down too.

7. Distribute the full Session 2 control sketch. Have students replace the placeholder MIN/MAX/CLAW constants at the top with their own recorded calibration numbers.
8. Upload, and walk through what changed versus Session 1: deadzone (ignoring joystick jitter near centre), incremental movement (slew) instead of instant snapping, constrained ranges (constrain()), and claw control by holding the second joystick's button.
9. Free-practice time: students pick up a pom-pom or small cube and set it down again, refining calibration numbers if the claw crushes objects or the arm still bumps its base.
10. Set up the challenge arena: three labelled bins/cups around the arm's reachable radius, and 12 pom-poms (4 red, 4 green, 4 blue) placed in a pickup zone in front of the arm.
11. Run timed rounds (90 seconds each): each student/pair scores 1 point per pom-pom placed in the correctly coloured bin, -0 for wrong bin (encourage retries, not penalties, to keep morale high), and bonus time-remaining points for finishing early.
12. Close with showcase: each group demonstrates one full pick-and-sort in front of the class, then reflects on what calibration change made the biggest difference.

Build detail

Materials for this session: everything from Session 1 (now fully wired) plus the 4th SG90 servo (claw), the claw/gripper linkage from the kit, extra jumper wires for the claw servo, coloured pom-poms/cubes (12), 3 labelled sorting bins, worksheet/ paper for calibration notes.

Wiring / assembly additions (component → pin):

- Claw servo signal → Arduino D6
- Claw servo red (power) wire → same shared servo power rail as other 3 servos
- Claw servo black/brown (ground) wire → same shared ground rail
- Joystick 2 SW pin (already wired to D3 in Session 1) now used in code to trigger claw open/close

```
/*
  Session 2a: Calibration Helper Sketch
  Robot Arm & Claw - TinkerWithMe
```

```
  Purpose: let students find safe MIN/MAX angles for each joint by
  typing single letters into the Serial Monitor to nudge one servo
  at a time, watching for the arm hitting its own frame.

  Commands (type into Serial Monitor, Enter after each):
    b+ / b-   nudge Base up/down by 5 degrees
    s+ / s-   nudge Shoulder up/down by 5 degrees
    e+ / e-   nudge Elbow up/down by 5 degrees
    c+ / c-   nudge Claw open/closed by 5 degrees
    p         print current angles for all four servos
*/

#include

const int PIN_SERVO_BASE     = 9;
const int PIN_SERVO_SHOULDER = 10;
const int PIN_SERVO_ELBOW    = 11;
const int PIN_SERVO_CLAW     = 6;

Servo baseServo, shoulderServo, elbowServo, clawServo;

int baseAngle = 90, shoulderAngle = 90, elbowAngle = 90, clawAngle = 45;

void setup() {
  Serial.begin(9600);
  baseServo.attach(PIN_SERVO_BASE);
  shoulderServo.attach(PIN_SERVO_SHOULDER);
  elbowServo.attach(PIN_SERVO_ELBOW);
  clawServo.attach(PIN_SERVO_CLAW);

  baseServo.write(baseAngle);
  shoulderServo.write(shoulderAngle);
  elbowServo.write(elbowAngle);
  clawServo.write(clawAngle);

  Serial.println("Calibration helper ready. Type commands: b+ b- s+ s- e+ e- c+ c- p");
}

void loop() {
  if (Serial.available() >= 2) {
    char joint = Serial.read();
    char dir = Serial.read();
    int step = (dir == '+') ? 5 : -5;

    if (joint == 'b') baseAngle = constrain(baseAngle + step, 0, 180);
    if (joint == 's') shoulderAngle = constrain(shoulderAngle + step, 0, 180);
    if (joint == 'e') elbowAngle = constrain(elbowAngle + step, 0, 180);
    if (joint == 'c') clawAngle = constrain(clawAngle + step, 0, 180);
    if (joint == 'p') {
      Serial.print("Base:"); Serial.print(baseAngle);
      Serial.print(" Shoulder:"); Serial.print(shoulderAngle);
      Serial.print(" Elbow:"); Serial.print(elbowAngle);
      Serial.print(" Claw:"); Serial.println(clawAngle);
    }

    baseServo.write(baseAngle);
    shoulderServo.write(shoulderAngle);
    elbowServo.write(elbowAngle);
    clawServo.write(clawAngle);
```

```
    // clear any leftover characters (like newline) from the input buffer
    while (Serial.available()) Serial.read();
  }
}
```

```
/*
  Session 2b: Full Calibrated Control Sketch
  Robot Arm & Claw - TinkerWithMe

  - Uses each joint's calibrated safe MIN/MAX (replace the placeholder
    numbers below with YOUR recorded calibration values).
  - Adds a deadzone so a resting joystick doesn't cause drift.
  - Adds slew-rate smoothing so the arm moves gradually and
    controllably instead of snapping to position.
  - Claw is controlled by holding Joystick 2's button (SW2):
    held down = grip closed, released = open.
*/

#include

// ---- Pin assignments ----
const int PIN_JOY1_X  = A0;  // Base
const int PIN_JOY1_Y  = A1;  // Shoulder
const int PIN_JOY2_X  = A2;  // Elbow
const int PIN_JOY2_SW = 3;   // Claw button (active LOW with INPUT_PULLUP)

const int PIN_SERVO_BASE     = 9;
const int PIN_SERVO_SHOULDER = 10;
const int PIN_SERVO_ELBOW    = 11;
const int PIN_SERVO_CLAW     = 6;

Servo baseServo, shoulderServo, elbowServo, clawServo;

// ---- REPLACE THESE WITH YOUR OWN CALIBRATED VALUES ----
const int BASE_MIN = 10,     BASE_MAX = 170;
const int SHOULDER_MIN = 40, SHOULDER_MAX = 160;
const int ELBOW_MIN = 20,    ELBOW_MAX = 150;
const int CLAW_OPEN = 60;    // angle when claw is fully open
const int CLAW_CLOSED = 5;   // angle when claw is gripping (not stalling!)

// current smoothed angles (float for fine incremental movement)
float curBase = 90, curShoulder = 90, curElbow = 90;

const int DEADZONE = 15;   // ignore small joystick jitter around centre (512)
const float STEP_SCALE = 0.3; // how much of the raw delta to apply each loop - lower = smoother/slower

void setup() {
  Serial.begin(9600);
  baseServo.attach(PIN_SERVO_BASE);
  shoulderServo.attach(PIN_SERVO_SHOULDER);
  elbowServo.attach(PIN_SERVO_ELBOW);
  clawServo.attach(PIN_SERVO_CLAW);

  pinMode(PIN_JOY2_SW, INPUT_PULLUP);

  // move to a safe home position before anything else happens
  baseServo.write((int)curBase);
  shoulderServo.write((int)curShoulder);
  elbowServo.write((int)curElbow);
```

```
  clawServo.write(CLAW_OPEN);
  delay(800);
}

// reads one joystick axis and returns a small movement step,
// ignoring readings inside the deadzone
int readAxisDelta(int pin) {
  int raw = analogRead(pin);         // 0-1023, centre approx 512
  int centered = raw - 512;
  if (abs(centered) < DEADZONE) return 0;
  return map(centered, -512, 512, -5, 5);
}

void loop() {
  // 1. read joystick movement for each of the 3 arm joints
  int dBase = readAxisDelta(PIN_JOY1_X);
  int dShoulder = readAxisDelta(PIN_JOY1_Y);
  int dElbow = readAxisDelta(PIN_JOY2_X);

  // 2. update target angles gradually (slew) and clamp to safe calibrated range
  curBase = constrain(curBase + dBase * STEP_SCALE, BASE_MIN, BASE_MAX);
  curShoulder = constrain(curShoulder + dShoulder * STEP_SCALE, SHOULDER_MIN, SHOULDER_MAX);
  curElbow = constrain(curElbow + dElbow * STEP_SCALE, ELBOW_MIN, ELBOW_MAX);

  baseServo.write((int)curBase);
  shoulderServo.write((int)curShoulder);
  elbowServo.write((int)curElbow);

  // 3. claw control - hold the button to grip, release to open
  bool gripping = (digitalRead(PIN_JOY2_SW) == LOW);
  clawServo.write(gripping ? CLAW_CLOSED : CLAW_OPEN);

  delay(20); // roughly 50 updates per second - smooth but responsive
}
```

Checks for understanding

- Why do we test each joint's safe range physically before hardcoding MIN/MAX into the sketch?
- What does the deadzone in `readAxisDelta()` actually fix, and what would happen without it?
- If the claw crushes a pom-pom flat, which single constant should you change, and in which direction?

Common mistakes & fixes

MISTAKE	SYMPTOM	FIX
Calibration MIN/MAX left as full 0–180	Arm crashes shoulder into base or elbow into shoulder bracket	Re-run calibration helper sketch, record tighter safe values, update constants

CLAW_CLOSED angle too aggressive	Claw servo buzzes/stalls and gets hot, or crushes soft objects	Increase CLAW_CLOSED angle slightly so it grips firmly but doesn't stall against itself
Deadzone value too small	Arm slowly drifts on its own even when joystick is untouched	Increase DEADZONE (e.g. from 15 to 25–30)
Claw button wired but code still uses old open/close logic from Session 1	Claw doesn't respond to the button at all	Confirm students copied the full Session 2b sketch, not a partial edit of Session 1's code

Assessment & Showcase

Assess students continuously against three checkpoints rather than a single end test. Checkpoint 1 (end of Session 1): the wiring-test sketch runs, every joystick axis visibly and correctly drives its matching servo, and all servos centre cleanly with no jitter. Checkpoint 2 (mid Session 2): the student has recorded sensible calibrated MIN/MAX/claw values on their worksheet and can demonstrate the arm moving through its full safe range without any part of the frame colliding. Checkpoint 3 (end of Session 2): the student can pick up at least one object and place it in a labelled bin during free practice, before the timed challenge even begins.

The final showcase is the timed pick-and-sort challenge run in front of the whole class. Each student/pair gets a 90-second round to sort as many of the 12 pom-poms into their correctly coloured bins as possible, using only their joystick controller. Score 1 point per correctly sorted object; incorrect placements can be retried within the same round. "Done" for this course means: the arm is fully assembled and screwed together securely, all four servos respond correctly and smoothly to the joystick controller, the arm does not collide with itself anywhere in its operating range, the claw can open and close reliably, and the student successfully sorts at least 3 of the 4 colours of objects during their showcase round. Facilitators should photograph or video each showcase round for the family/parent share-out.

Extension & Home Challenges

- **Easy:** Add a "home position" shortcut — when both joystick buttons (SW1 and SW2) are pressed at the same time, snap the arm back to base=90, shoulder=90, elbow=90, claw=open. Great first step into combining digital reads.
- **Medium:** Wire the previously unused Joystick 2 Y-axis (A3) to a 5th servo mounted as a wrist-rotation joint on the claw, turning this into a true 4-DOF arm. Requires re-mounting the claw onto a rotating wrist bracket and adding a 5th calibrated MIN/MAX pair to the code.
- **Hard:** Replace manual joystick sorting with a semi-autonomous "macro" mode: store 3–4 waypoints (arrays of base/shoulder/elbow/claw angles) for "hover over pickup zone," "lower and grip," "lift and move to bin," and "release," then trigger the whole sequence with a single button press using `millis()`-based timing instead of `delay()`, so the robot performs one full pick-and-place cycle on its own.